



YAOUNDE INTERNATIONAL BUSINESS SCHOOL
Bachelor of Technology (B.Tech)

Course Code & Title:	SWEY4109: Introduction to Data Science
Department:	Software Engineering
Credit & Credit Hours:	6 Credits
Lecturer:	MUNJAM Thomas T. (MSc. Computer Engineering)

Tutorial 101

This exercise will test your ability to read a data file and understand statistics about the data.

In subsequent parts of the exercise, you will apply techniques to filter the data, build a machine learning model, and iteratively improve your model.

The course examples which we have seen used data from Melbourne. To ensure you can apply the techniques we have learnt on your own, you will have to apply them to a new dataset (with house prices from Iowa).

For this first tutorial, you will be given step by step directives on what you have to do. Make sure to use knowledge from the lessons learnt to accomplish this task.

SUBMISSION INSTRUCTIONS

The task must be completed on or before Sunday 11th January 2026 at 11:59PM

Prepare a PDF document which bears your name and campus. It will contain screenshots of your compilation outputs and necessary explanations/interpretations. You will submit a **.zip** file, containing this PDF document alongside your python file to the to the link <https://www.personlinks.org/assignments> respecting the above deadline. LATE SUBMISSIONS WILL NOT BE CONSIDERED.

INTRODUCTION TO DATA SCIENCE WITH PYTHON TUTORIAL 101

PHASE 1

Step 1: Loading Data

Read the Iowa data file into a Pandas DataFrame called *home_data*.

Step 2: Review The Data

Use the command you learned to view summary statistics of the data. Then fill in variables to answer the following questions

- What is the average lot size (rounded to nearest integer)?
- As of today, how old is the newest home (current year - the date in which it was built)
- Write out a command to print out your answers

Step 3: Think About Your Data (Non-coding)

The newest house in your data isn't that new. A few potential explanations for this:

1. They haven't built new houses where this data was collected.
2. The data was collected a long time ago. Houses built after the data publication wouldn't show up.

If the reason is explanation #1 above, does that affect your trust in the model you build with this data? What about if it is reason #2?

How could you dig into the data to see which explanation is more plausible?

PHASE 2

So far, you have loaded your data and reviewed it.

Step 1: Specify Prediction Target

Select the target variable, which corresponds to the sales price. Save this to a new variable called *y*. You'll need to print a list of the columns to find the name of the column you need.

Step 2: Create X

Now you will create a DataFrame called *X* holding the predictive features.

Since you want only some columns from the original data, you'll first create a list with the names of the columns you want in **X**.

You'll use just the following columns in the list:

- LotArea
- YearBuilt
- 1stFlrSF
- 2ndFlrSF
- FullBath
- BedroomAbvGr
- TotRmsAbvGrd

After you've created that list of features, use it to create the DataFrame that you'll use to fit the model.

Step 3: Review Data (Non-coding)

Before building a model, take a quick look at **X** to verify it looks sensible.

Step 4: Specify and Fit Model

Create a `DecisionTreeRegressor` and save it *iowa_model*. Ensure you've done the relevant import from *sklearn* to run this command. Then fit the model you just created using the data in **X** and **y** that you saved above.

NOTE: For model reproducibility, set a numeric value for *random_state* when specifying the model.

Step 5: Make Predictions

Make predictions with the model's `predict` command using **X** as the data. Save the results to a variable called `predictions`.

Think About Your Results

Use the `head` method to compare the top few predictions to the actual home values (in **y**) for those same homes. Anything surprising?

It's natural to ask how accurate the model's predictions will be and how you can improve that. That will be your next step.

PHASE 3

Recap

You've built a model. In this exercise you will test how good your model is.

Step 1: Split Your Data

Use the `train_test_split` function to split up your data.

Give it the argument `random_state=1` so the check functions know what to expect when verifying your code. Recall, your features are loaded in the DataFrame **X** and your target is loaded in **y**.

Step 2: Specify and Fit the Model

Create a `DecisionTreeRegressor` model and fit it to the relevant data. Set `random_state` to 1 again when creating the model.

Step 3: Make Predictions with Validation data

Predict with all validation observations

Inspect your predictions and actual values from validation data.

- print the top few validation predictions
- print the top few actual prices from validation data

Explain what you notice that is different from what you saw with in-sample predictions.

Try to remember why validation predictions differ from in-sample (or training) predictions?

Step 4: Calculate the Mean Absolute Error in Validation Data

Check if your MAE good? There isn't a general rule for what values are good that applies across applications.

PHASE 4

Recap

You should write the function `get_mae` yourself.

Step 1: Compare Different Tree Sizes

Write a loop that tries the following values for `max_leaf_nodes` from a set of possible values.

Call the `get_mae` function on each value of `max_leaf_nodes`. Store the output in some way that allows you to select the value of `max_leaf_nodes` that gives the most accurate model on your data.

Step 2: Fit Model Using All Data

You know the best tree size. If you were going to deploy this model in practice, you would make it even more accurate by using all of the data and keeping that tree size. That is, you don't need to hold out the validation data now that you've made all your modeling decisions.

You've tuned this model and improved your results. But we are still using Decision Tree models, which are not very sophisticated by modern machine learning standards. In the next step you will learn to use Random Forests to improve your models even more.

PHASE 5

Data science isn't always this easy. But replacing the decision tree with a Random Forest is seen to be an easy win.

This phase requires you to simply use random forests as opposed to decision tree on the provided dataset.